public static void main(String[] args) {
// TODO Auto-generated method stub
int n = 4;
int tq = 2;// total Queen
boolean[] baord = new boolean[n];
Permutation(baord, tq,

"", 0);

}

public static void Permutation(boolean[] baord, int tq, String ans, int qpsf) { if (qpsf == tq) {
System.out.println(ans);
return;
}

for (int i = 0; i < baord.length; i++) {
if (baord[i] == false) {
baord[i] = true;
Permutation(baord, tq, ans + "b" + i + "q" + qpsf, qpsf + 1);
}

}

}

public static void main(String[] args) {
// TODO Auto-generated method stub
int n = 4;
int tq = 2;// total Queen
boolean[] baord = new boolean[n];
Permutation(baord, tq,

"", 0);

}

public static void Permutation(boolean[] baord, int tq, String ans, int qpsf) { if (qpsf == tq) {
System.out.println(ans);
return;
}

for (int i = 0; i < baord.length; i++) {
if (baord[i] == false) {
baord[i] = true;
Permutation(baord, tq, ans + "b" + i + "q" + qpsf, qpsf + 1); baord[i] = false;
}

}

}

public static void main(String[] args) {
// TODO Auto-generated method stub
int n = 4;
int tq = 2;// total Queen
boolean[] baord = new boolean[n];
Combination(baord, tq,

"", 0,0);

}

public static void Combination(boolean[] baord, int tq, String ans, int qpsf,int idx) { if (qpsf == tq) {
System.out.println(ans);
return;
}

for (int i = idx; i < baord.length; i++) {
if (baord[i] == false) {
baord[i] = true;
Combination(baord, tq, ans + "b" + i + "q" + qpsf, qpsf + 1,i+1); baord[i] = false;
}

}

}

public static void main(String[] args) {
int[] coin = { 2, 3, 5 };
int ammount = 8;
Permutation(coin, ammount,

"");

}

public static void Permutation(int[] coin, int ammount, String ans) { // TODO Auto-generated method stub
if (ammount == 0) {
System.out.println(ans);
return;
}
for (int i = 0; i < coin.length; i++) {
if (ammount >= coin[i]) {
// ammount -= coin[i];
Permutation(coin, ammount - coin[i], ans + coin[i]);
// ammount += coin[i];

}

}

}

public static void main(String[] args) {
int[] coin = { 2, 3, 5 };
int ammount = 8;
List<Integer> ll = new ArrayList<>();
Combination(coin, ammount, ll, 0);

}

public static void Combination(int[] coin, int ammount, List<Integer> ll, int idx) { // TODO Auto-generated method stub
if (ammount == 0) {
System.out.println(ll);
return;
}
for (int i = idx; i < coin.length; i++) {
if (ammount >= coin[i]) {
// ammount -= coin[i];
ll.add(coin[i]);
Combination(coin, ammount - coin[i],ll, i);
// ammount += coin[i];

}

}

}

public static void Combination(int[] coin, int ammount, List<Integer> ll, int
idx,List<List<Integer>> ans) {
// TODO Auto-generated method stub
if (ammount == 0) {
//System.out.println(ll);
ans.add(ll);
return;
}
for (int i = idx; i < coin.length; i++)
{
if (ammount >= coin[i]) {
// ammount -= coin[i];
ll.add(coin[i]);
Combination(coin, ammount -
coin[i], ll, i,ans);
// ammount += coin[i];
ll.remove(ll.size() - 1);

}

}

}

public static void Combination(int[] coin, int ammount, List<Integer> ll, int idx,List<List<Integer>> ans) { // TODO Auto-generated method stub
if (ammount == 0) {
//System.out.println(ll);
ans.add(ll);
return;
}
for (int i = idx; i < coin.length; i++) {
if (ammount >= coin[i]) {
// ammount -= coin[i];
ll.add(coin[i]);
Combination(coin, ammount - coin[i], ll, i,ans); // ammount += coin[i];
ll.remove(ll.size() - 1);

}

}

}

Lec-18

08/03/26 4:43 PM